



Structured Previews

Structured Previews let Klogg **decode and parse a single log line** into a **tree of named fields** using a JSON configuration file. This is useful for logs that contain embedded protocols, hex/base64 blobs, bitmasks, CRCs, etc.

Previews are **on-demand** (they do not modify your log file) and are shown in a **non-modal Preview window** that supports **tabs**:

- Each **Send to Preview** action opens a **new tab** (one message per tab, until user closes it).
 - Each tab includes:
 - a **Preview Type** combo (Auto + all preview definitions)
 - a structured preview output area (tree/table)
 - Preview type is **auto-selected** by regex match, but can be overridden manually.
-

Table of contents

- [Structured Previews](#)
 - [Table of contents](#)
 - [Quick start](#)
 - [Preview configuration file](#)
 - [Schema & editor autocomplete](#)
 - [Top-level config structure](#)
 - [Top-level keys](#)
 - [Preview definition](#)
 - [Keys](#)
 - [Field definition](#)
 - [Keys](#)
 - [Reusable blocks](#)
 - [Types](#)
 - [Hex odd-length notes](#)

- [Formats](#)
 - [Expressions and variables](#)
 - [Variable rules](#)
 - [Capture sources](#)
 - `source: "capture"`
 - `source: "literal"`
 - [Match stage](#)
 - [Examples](#)
 - [Example 1: Minimal preview](#)
 - [Example 2: EHCP \(direct\)](#)
 - [Example 3: SRING → hex decode → EHCP \(two-stage\)](#)
 - [Example 4: Enum, flags, bitfield](#)
 - [Example 5: Nested base64 field](#)
 - [Importing previews in the UI](#)
 - [Troubleshooting](#)
 - [“Missing property capture” in editor](#)
 - [Decode errors](#)
 - [Regex mismatch inside match stage](#)
 - [Expression errors](#)
 - [Notes](#)
-

Quick start

1. Create a JSON file (e.g. `previews.json`) using the schema:

```
{
  "$schema": "../schemas/klogg-previews.schema.json",
  "version": 1,
  "previews": []
}
```

2. Add one or more preview definitions in `previews[]`.
3. In Klogg:
 - **Tools** → **Import previews...**

- Select your JSON file
4. Right-click a log line:
- **Send to Preview** → **Auto** (or pick a specific preview)
 - A Preview window opens (non-modal)
 - The parsed output appears in a new tab
-

Preview configuration file

Schema & editor autocomplete

Klogg ships a JSON schema at:

- `schemas/klogg-previews.schema.json`

To enable validation/autocomplete in editors (VS Code / JetBrains etc.), add at the root:

```
"$schema": "../schemas/klogg-previews.schema.json"
```

If your editor cannot resolve relative schema paths, replace it with an absolute path on your machine.

Top-level config structure

```
{
  "$schema": "../schemas/klogg-previews.schema.json",
  "version": 1,
  "previews": [
    {
      "name": "Example",
      "enabled": true,
      "regex": "^(?<payload>.*)$",
      "bufferCapture": "payload",
      "type": "string",
      "format": "fields",
      "fields": []
    }
  ]
}
```

Top-level keys

Key	Type	Required	Meaning
<code>\$schema</code>	string	no	Path/URL to JSON schema (autocomplete/validation)
<code>version</code>	int	no	Config version (currently <code>1</code>)
<code>previews</code>	array	yes	List of preview definitions

Preview definition

A preview definition matches a log line (regex), optionally selects a capture as the parsing buffer, optionally decodes it (type), then parses it using `fields`.

Keys

Key	Type	Required	Meaning
<code>name</code>	string	yes	Preview name shown in UI
<code>regex</code>	string	yes	Regex applied to the original log line
<code>enabled</code>	bool	no	Include in Auto/members list; default <code>true</code>
<code>priority</code>	int	no	If multiple previews match, higher priority can win (if supported)
<code>bufferCapture</code>	string/ int	no	Capture (name or index) used as “buffer” for buffer-based parsing; default capture <code>0</code> (full match)
<code>offset</code>	int/ string	no	Initial skip into buffer before parsing fields (default <code>0</code>)
<code>type</code>	string	no	Buffer encoding/type (see Types); default <code>"string"</code>
<code>format</code>	string	yes	Currently <code>"fields"</code> at preview-level
<code>fields</code>	array	yes	Field definitions

Field definition

Fields are parsed in order (like a cursor walking through a buffer), unless a field’s `source` is `"capture"` or `"literal"`.

Keys

Key	Type	Required	Meaning
<code>name</code>	string	yes	Field name; also used for <code>{var}</code> expressions

Key	Type	Required	Meaning
source	string	no	"buffer" (default), "capture", or "literal"
capture	string/ int	conditional	Required if <code>source == "capture"</code> . Capture name/index in current match context
value	string	conditional	Required if <code>source == "literal"</code> . Constant value
offset	int/ string	no	Skip relative to current cursor before taking field value
width	int/ string	no	Length of field slice
type	string	no	Encoding/type of the slice before formatting (see Types)
endianness	string	no	"little" or "big" for numeric interpretation of multi-byte values
format	string	no	Rendering/parsing mode (see Formats)
enumMap	object	conditional	Required when <code>format == "enum"</code>
flagMap	object	conditional	Required when <code>format == "flags"</code>
bitfieldMap	array	conditional	Required when <code>format == "bitfield"</code>
regex	string	conditional	Required when <code>format == "match"</code>
bufferCapture	string/ int	no	For <code>format == "match"</code> : which capture becomes the nested buffer for child parsing
fields	array	conditional	Required when <code>format == "fields"</code> or <code>format == "match"</code>

Reusable blocks

Reusable blocks let you define field groups once and invoke them by name.

- Define blocks in top-level `blocks[]` (each entry is a field definition with a required `name`).
- Invoke a block by setting `format: "block"` (or `source: "block"`) and a `block` string.
- `block` supports `{var}` placeholders, resolved from raw parsed values (not enum labels).

Example:

```
{
  "blocks": [
    {
      "name": "payload_P_CM",
      "format": "fields",
      "fields": [
        { "name": "payload", "width": 3, "type": "string", "format": "string" }
      ]
    }
  ],
  "previews": [
    {
      "name": "Block dynamic",
      "regex": "^(?<payload>.*)$",
      "bufferCapture": "payload",
      "format": "fields",
      "fields": [
        { "name": "protocol_type", "width": 1, "format": "string" },
        { "name": "command_id", "width": 2, "format": "string" },
        { "name": "payload_block", "format": "block",
          "block": "payload_{protocol_type}_{command_id}" }
      ]
    }
  ]
}
```

If a block is missing or a cycle is detected, the preview shows an error node and continues.

Types

`type` describes how the buffer/slice is **represented** and/or how it must be **decoded** before formatting.

Supported types:

Type	Meaning
<code>string</code>	Text buffer (no decoding)
<code>hexString</code>	A string of hex digits (e.g. <code>D774</code> , <code>042</code> , <code>4A4B4C</code>)
<code>base64</code>	Base64 encoded string; decode to bytes before parsing/formatting
<code>bin</code> / <code>binary</code> / <code>bytes</code>	Raw binary data (byte buffer)

Hex odd-length notes

For **numeric** formats (`dig` / `dec` / `hex` / `bin` / `enum` / `flags` / `bitfield`), `hexString` supports **odd digit counts**. Example:

- `"042"` is valid and means `0x042 = 66`

For **byte-decoding** use cases (turning `hexString` into bytes), odd-length strings may be left-padded with `0` to decode consistently.

Formats

`format` describes how to **render** or **parse** the (decoded) slice.

Supported formats:

Format	Meaning
<code>string</code>	Display as text
<code>dig /</code> <code>dec</code>	Display numeric value in decimal
<code>hex</code>	Display numeric value in hex
<code>bin</code>	Display numeric value in binary
<code>strlen</code>	Display string length
<code>enum</code>	Map value using <code>enumMap</code>
<code>flags</code>	Interpret numeric value as bitmask using <code>flagMap</code>
<code>bitfield</code>	Split numeric value into subfields described by <code>bitfieldMap</code>
<code>fields</code>	Treat slice as a buffer and parse nested fields sequentially
<code>match</code>	Decode → apply regex → create new capture/buffer context → parse nested fields

Expressions and variables

`width` and `offset` support:

- integer literals: `3`, `12`
- expressions: `"{size}-5"`, `"{len}+1"`

Variable rules

- Variables refer to **previously parsed fields by name** in the current parsing scope.
- Variables resolve to the **numeric value** of a field.
- If a variable is missing or non-numeric, the dependent field should show an error (and logs should explain).

Supported expression grammar is intentionally small and safe:

- `{var}`
 - `{var} + int` , `{var} - int`
 - `{var1} + {var2}` , `{var1} - {var2}`
-

Capture sources

By default, fields parse from the **current buffer** (`source: "buffer"`). You can also pull values from regex captures or constant literals.

source: "capture"

Use this when you want to display a value captured by regex rather than slicing the buffer.

```
{
  "name": "checksum",
  "source": "capture",
  "capture": "checksum",
  "type": "hexString",
  "format": "hex"
}
```

`capture` is required only when `source` is `"capture"` .

source: "literal"

```
{
  "name": "Protocol",
  "source": "literal",
  "value": "EHCP",
  "format": "string"
}
```

Match stage

`format: "match"` is the recommended way to implement **multi-stage parsing**:

1. Take the field input (buffer slice or capture)
2. Decode it using `type` (optional)
3. Apply `regex` to the decoded text
4. Create a new “match context”:
 - captures from this regex are available to child fields using `source: "capture"`
 - the nested parsing buffer is:
 - `bufferCapture` capture from this match, if specified, OR
 - capture `0` (full match) by default
5. Parse child fields from that nested buffer (positional parsing) or from captures

This supports wrapper formats like:

- SRING wrapper → extract payload → hex decode → parse EHCP inside

Examples

Example 1: Minimal preview

Parses the whole line as a single `text` field.

```
{
  "$schema": "../schemas/klogg-previews.schema.json",
  "version": 1,
  "previews": [
    {
      "name": "Raw line",
      "enabled": true,
      "regex": "^(?<line>.*)$",
      "bufferCapture": "line",
      "type": "string",
      "format": "fields",
      "fields": [
        { "name": "text", "format": "string" }
      ]
    }
  ]
}
```

Example 2: EHCP (direct)

Log line (example):

```
12/10/2025 12:57:45.945 [RX] - #4 "EHCP042020901004104108001078251210105737E#INN...!D774"
```

Config:

```

{
  "$schema": "../schemas/klogg-previews.schema.json",
  "version": 1,
  "previews": [
    {
      "name": "EHCP (quoted)",
      "enabled": true,
      "regex": "^.*\\\\"(?<ehcp>EHCP[^\\""]*)\\\\".*$",
      "bufferCapture": "ehcp",
      "type": "string",
      "format": "fields",
      "fields": [
        {
          "name": "ehcp",
          "format": "match",
          "regex": "^(?<header>EHCP)(?<payload>[^\"]*)(?<checksum>[:xdigit:]{4})$",
          "bufferCapture": "payload",
          "fields": [
            {
              "name": "header",
              "source": "capture",
              "capture": "header",
              "format": "string"
            },
            {
              "name": "size",
              "width": 3,
              "type": "hexString",
              "format": "dig"
            },
            {
              "name": "data",
              "width": "{size}-5",
              "format": "string"
            },
            {
              "name": "checksum",
              "source": "capture",
              "capture": "checksum",

```

```
        "type": "hexString",
        "format": "hex"
      }
    ]
  }
]
```

Example 3: SRING → hex decode → EHCP (two-stage)

SRING line:

```
SRING: 1,48,4548435030323930303030...
```

Decoded payload becomes:

```
EHCP029000000004104108001000251210105746Z#0!C40D
```

Config:

```

{
  "$schema": "../schemas/klogg-previews.schema.json",
  "version": 1,
  "previews": [
    {
      "name": "SRING → EHCP",
      "enabled": true,
      "regex": "^SRING:\\s*\\d,\\d\\d,(?<hexPayload>[0-9A-Fa-f]+)$",
      "bufferCapture": "hexPayload",
      "type": "string",
      "format": "fields",
      "fields": [
        {
          "name": "ehcp",
          "format": "match",
          "type": "hexString",
          "regex": "^(?<header>EHCP)(?<payload>[^\"]*)(?<checksum>[[:xdigit:]]{4})$",
          "bufferCapture": "payload",
          "fields": [
            {
              "name": "header",
              "source": "capture",
              "capture": "header",
              "format": "string"
            },
            {
              "name": "size",
              "width": 3,
              "type": "hexString",
              "format": "dig"
            },
            {
              "name": "data",
              "width": "{size}-5",
              "format": "string"
            },
            {
              "name": "checksum",
              "source": "capture",

```

```
        "capture": "checksum",
        "type": "hexString",
        "format": "hex"
    }
]
}
]
}
```

Example 4: Enum, flags, bitfield

```
{
  "$schema": "../schemas/klogg-previews.schema.json",
  "version": 1,
  "previews": [
    {
      "name": "FlagsEnumExample",
      "enabled": true,
      "regex": "^Flag (?<idx>\\d): Value=(?<val>\\d+)$",
      "type": "string",
      "format": "fields",
      "fields": [
        {
          "name": "FlagIndex",
          "source": "capture",
          "capture": "idx",
          "format": "dig"
        },
        {
          "name": "ValueType",
          "source": "capture",
          "capture": "val",
          "format": "enum",
          "enumMap": {
            "0": "OFF",
            "1": "ON",
            "2": "UNKNOWN"
          }
        },
        {
          "name": "Permissions",
          "source": "capture",
          "capture": "val",
          "format": "flags",
          "flagMap": {
            "0x1": "READ",
            "0x2": "WRITE",
            "0x4": "EXECUTE",
            "0x8": "DELETE"
          }
        }
      ]
    }
  ]
}
```

```
}  
  ]  
}  
]  
}
```

Example 5: Nested base64 field

```
{
  "$schema": "../schemas/klogg-previews.schema.json",
  "version": 1,
  "previews": [
    {
      "name": "Nested base64 payload",
      "enabled": true,
      "regex": "^(?<payload>[A-Za-z0-9+/=]+)$",
      "bufferCapture": "payload",
      "type": "string",
      "format": "fields",
      "fields": [
        {
          "name": "size",
          "width": 4,
          "type": "hexString",
          "format": "dig"
        },
        {
          "name": "value",
          "width": "{size}",
          "type": "base64",
          "format": "fields",
          "fields": [
            { "name": "subValue1", "width": 2, "type": "bytes", "endianness": "little", "format": "hex" },
            { "name": "subValue2", "width": 2, "type": "bytes", "endianness": "little", "format": "hex" },
            { "name": "url", "type": "string", "format": "string" }
          ]
        }
      ]
    }
  ]
}
```

Importing previews in the UI

1. Open **Tools** → **Import previews...**
2. Select your JSON file
3. Import dialog shows a table with:
 - Name (read-only)
 - Pattern/Regex (read-only)
 - Enabled (checkbox)
4. Click **Import**

After import, previews appear in:

- Preview Type dropdown (Auto + enabled previews)
 - Right-click **Send to Preview** menu (Auto + enabled preview list)
-

Troubleshooting

“Missing property capture” in editor

`capture` is required **only** when you use:

```
"source": "capture"
```

Otherwise it should be optional.

Decode errors

Common causes:

- hexString contains non-hex chars (quotes, separators)
- base64 contains invalid chars or padding
- wrong capture group: you captured too much (like `"...!D774"` including `!`)

Fixes:

- tighten the regex group for payload
- use `offset` to skip delimiters (e.g. skip `!`)
- validate the extracted string by printing it in preview output first

Regex mismatch inside match stage

If `format: "match"` doesn't match decoded text:

- verify anchors `^...$`
- verify decoded buffer is ASCII/UTF-8
- check logs for the decoded snippet and regex used

Expression errors

If `{size}` is missing:

- ensure `size` is parsed earlier in the same scope
 - ensure `size` is numeric (`format: "dig"/"hex"/"bin"/etc.`)
-

Notes

- Previews are designed to parse **fields of any length** as long as you can express it with:
 - fixed `width / offset`
 - `{vars}` expressions
 - nested `match` stages for multi-step decode+regex parsing
- Previews work alongside filters/highlighters/live tailing (no special restrictions).